Project Phoenix Autocallable - Pricing & Hedging under Heston model

Author : Baptiste SOREL

Date : February 2026

Section 1: Heston Model & the parameters

For this project, I implemented the Heston Model. Unlike Black-Scholes, Heston assumes that volatility itself is a random process. This allows for a more realistic modeling of market risks, particularly the probability of hitting downside barriers ("Fat Tails").
While the Local Volatility model (Dupire) creates a perfect fit for the current vanilla option surface, it assumes that volatility is a deterministic function of the spot price $\sigma(S,t)$. This deterministic nature often leads to unrealistic "Forward Skew" dynamics, typically flattening the skew for future maturities.For a path-dependent product like the Phoenix Autocallable, the pricing depends heavily on conditional probabilities of hitting barriers in the future. The Heston model, by treating volatility as a stochastic process, better captures the vol of vol, which impacts the cost of the barrier protection, and a realistic forward skew over time, which is crucial for pricing accurately the PDI.

Heston model :

$$dS_t = \mu S_t dt + \sqrt{v_t} S_t dW_t$$
$$dv_t = \kappa(\theta - v_t)dt + \sigma\sqrt{v_t} dZ_t$$

Where the Brownian motions dW_t and dZ_t have correlation ρ. The key parameters are :

- κ : mean reversion speed
- θ : Long term average variance (mean reversion)
- σ : volatility of volatility
- ρ : Correlation between the asset price and its volatility. In equity, it is generally negative, creating the skew. As the underlying is the S&P500 (SPY), it is the case here.
- v_0 : Initial variance

Note on calibration

In a live trading environment, these parameters are "calibrated" to fit the market prices of vanilla options (Calls and Puts). I tried to develop a calibration engine (available in `src/Models/calibration.py`) using the Gil-Pelaez Fourier transform method combined with a least-squares minimization algorithm

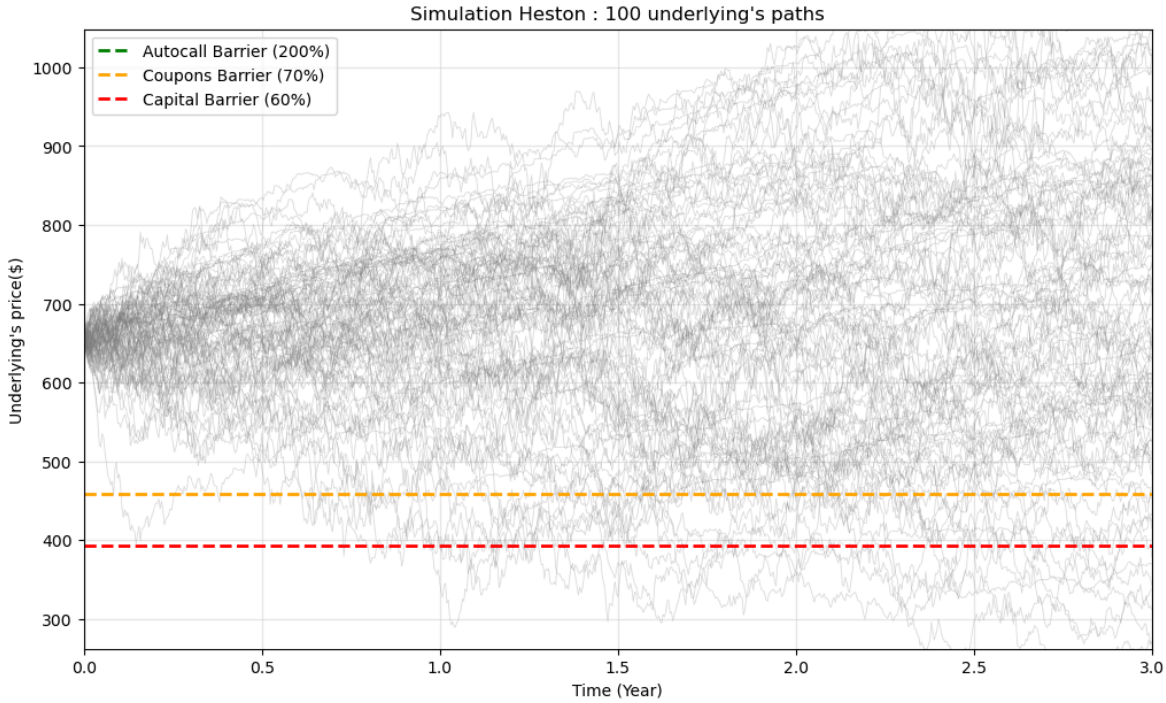
(scipy.optimize) to minimize the error between the model and market data fetched from Yahoo Finance.

However, the calibration didn't worked properly. Due to data granularity issues and the instability of the optimization process on free data, the automated calibration yielded unstable results. To prioritize the stability of the Pricing and Hedging simulation (the core of this project), I chose to fix the parameters to realistic market values (e.g., $\rho = -0.7$ for S&P 500 behavior) rather than relying on an unstable calibration output.

In [151…

```
%load_ext autoreload
%autoreload 2

from src.Models import calibration
from src.Models.Heston import HestonModel
import matplotlib.pyplot as plt
import numpy as np

np.random.seed(1)

nb_paths_to_plot=100
S0=calibration.S0
N_year=252
r=calibration.r
T=calibration.T
N=10**4
model=HestonModel(kappa=calibration.kappa,theta=calibration.theta,sigma=c

total_steps=int(T*N_year)
paths=model.simulate_paths(N,total_steps)
subset_paths=paths[:, :nb_paths_to_plot]

time_axis=np.linspace(0, T, subset_paths.shape[0])

plt.figure(figsize=(12, 7))
plt.plot(time_axis, subset_paths, color='grey', alpha=0.3, linewidth=0.5)

plt.axhline(y=calibration.barrier_autocall * S0, color='green', linestyle
plt.axhline(y=calibration.barrier_coupon * S0, color='orange', linestyle=
plt.axhline(y=calibration.barrier_capital * S0, color='red', linestyle='-

plt.title(f"Simulation Heston : {nb_paths_to_plot} underlying's paths")
plt.xlabel("Time (Year)")
plt.ylabel("Underlying's price($)")
plt.xlim(0, T)
plt.ylim(S0 * 0.4, S0 * 1.6)

plt.legend(loc='upper left')
plt.grid(True, alpha=0.3)
```

```
The autoreload extension is already loaded. To reload it, use:
  %reload_ext autoreload
```



Section 2: Structuration & Pricing

Here are the parameters for the rest of the project :

```
ticker=yf.Ticker("SPY")
S0=ticker.history(period="1d")["Close"].iloc[-1]
r=yf.Ticker("^IRX").history(period="1d")["Close"].iloc[-1]
/ 100 #The risk-free rate
T=3
dt=0.25 # Frequency of observations : every quarter
coupon_rate=0.025 # We start with a coupon of 2.5% every
quarter
barrier_autocall=1
barrier_coupon=0.7
barrier_capital=0.6
nominal=1000


v0=0.04        # current variance
kappa=2.0      #mean reversion speed
theta=0.04     #Long term variance, mean reversion
sigma=0.5      #vol of the volatility
rho=-0.7       #Correlation between the underlying and its
vol
```

The spot price S_0 is set to the latest closing price of the SPY ETF (S&P 500 Proxy).

The rate r is derived from the CBOE Interest Rate 13-Week T-Bill (IRX). This ensures that the discounting and the drift in the Heston model are consistent with the current monetary environment.

We structured a classic Phoenix Autocallable, with a $T = 3$ years maturity, an autocallable barrier at 100%, a coupon barrier at 70%, and a capital barrier at 60%.

Initally, we consider of coupon of 2% quarterly (8%), that we may change later if the margin is negative for the seller (the bank).

In [152…

```
from src.Products.phoenix import Phoenix

calibration.coupon_rate=0.02

phoenix=Phoenix(T=T,dt=calibration.dt,coupon_rate=calibration.coupon_rate

N=10**4

model=HestonModel(kappa=calibration.kappa,theta=calibration.theta,sigma=c

total_steps=int(T*N_year)
paths=model.simulate_paths(N,total_steps)

client_payoffs=phoenix.evaluate_payoffs_prices(paths)[0]
product_prices=phoenix.evaluate_payoffs_prices(paths)[1]

mean_payoffs=np.mean(client_payoffs)
std_payoffs=np.std(client_payoffs)
mean_prices=np.mean(product_prices)
std_prices=np.std(product_prices)

error_margin = 1.96 * (std_prices / np.sqrt(N))


#### RESULTS ########
print("The average price for this product is "+str(round(mean_prices,2))+
if mean_prices<calibration.nominal:
    print("Positive margin : "+str(round(calibration.nominal-mean_prices,
else:
    print("Negative margin : "+str(round(calibration.nominal-mean_prices,
```

```
The average price for this product is 1062.26 with a 95% confidence interv
al : [1059.0,1065.53]
Negative margin : -62.26$
```

When pricing with a coupon of 8%, the margin is negative. It is still quite uncertain due to the statistical noise of the Monte Carlo simulation.

To raise our margin, one can lower our coupon rate. Let's apply `calibration.coupon_rate=0.015`

In [153…

```
from src.Products.phoenix import Phoenix

calibration.coupon_rate=0.01125

phoenix=Phoenix(T=T,dt=calibration.dt,coupon_rate=calibration.coupon_rate

client_payoffs=phoenix.evaluate_payoffs_prices(paths)[0]
product_prices=phoenix.evaluate_payoffs_prices(paths)[1]

mean_payoffs=np.mean(client_payoffs)
std_payoffs=np.std(client_payoffs)
mean_prices=np.mean(product_prices)
std_prices=np.std(product_prices)
```

```
error_margin=1.96 * (std_prices / np.sqrt(N))


#### RESULTS ########
print("The average price for this product is "+str(round(mean_prices,2))+
if mean_prices<calibration.nominal:
    print("Positive margin : "+str(round(calibration.nominal-mean_prices,
else:
    print("Negative margin : "+str(round(calibration.nominal-mean_prices,
```

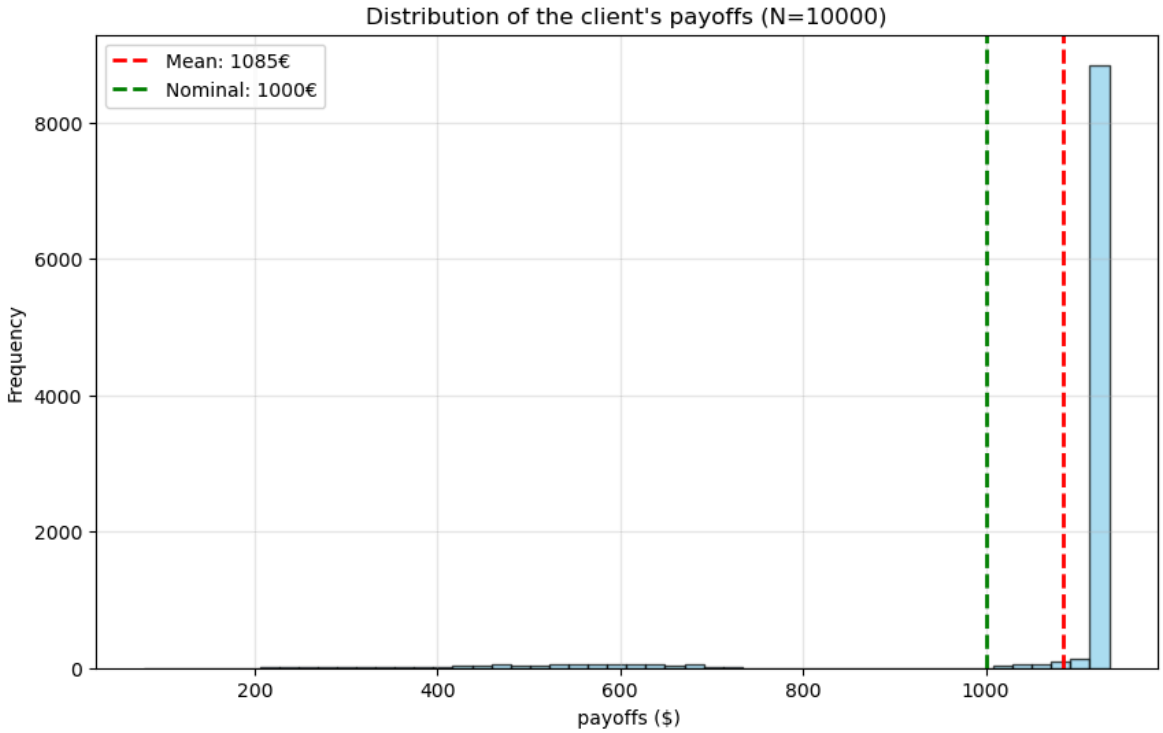
```
The average price for this product is 973.47 with a 95% confidence interva
l : [970.48,976.46]
Positive margin : 26.53$
```

We select the 1.25% coupon structure. This +26.5 margin (2.65% of the notionnal) provides a necessary safety buffer to cover hedging transaction costs.

Let's now plot the distribution of the client's payoff with our product, for $N = 10^4$ simulations :

In [154…

```
plt.figure(figsize=(10, 6))
plt.hist(client_payoffs, bins=50, color='skyblue', edgecolor='black', alp
plt.axvline(mean_payoffs, color='red', linestyle='dashed', linewidth=2, l
plt.axvline(1000, color='green', linestyle='dashed', linewidth=2, label='

plt.title(f"Distribution of the client's payoffs (N={N})")
plt.xlabel(" payoffs ($)")
plt.ylabel("Frequency")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```



The distribution reveals a highly concentrated profile, with approximately 87% of outcomes clustered above the Nominal value at ~

$1085, correspondingtoAutocallscenarioswheretheinvestorreceivesthecapitalpl$
$.Thisconcentrationatasinglepayoffleveleflectsthecurrentmarketenvironme$
$-neutraldrift, theunderlyingsystematicallyreachestheAutocallBarrier(100d$
$.Themeanpayoffof$
1085 significantly exceeds the Nominal, confirming the negative margin for the bank observed in the pricing section. The long left tail captures the residual downside risk, representing paths where sustained volatility breaches the Capital Barrier (60%) and erodes the investor's principal.

Section 3: PnL simulation & Delta hedging

We will now simulate the life of the product, and simulate the PnL of the bank from selling this product. In addition, we will also create a delta neutral portfolio and simulates its PnL.

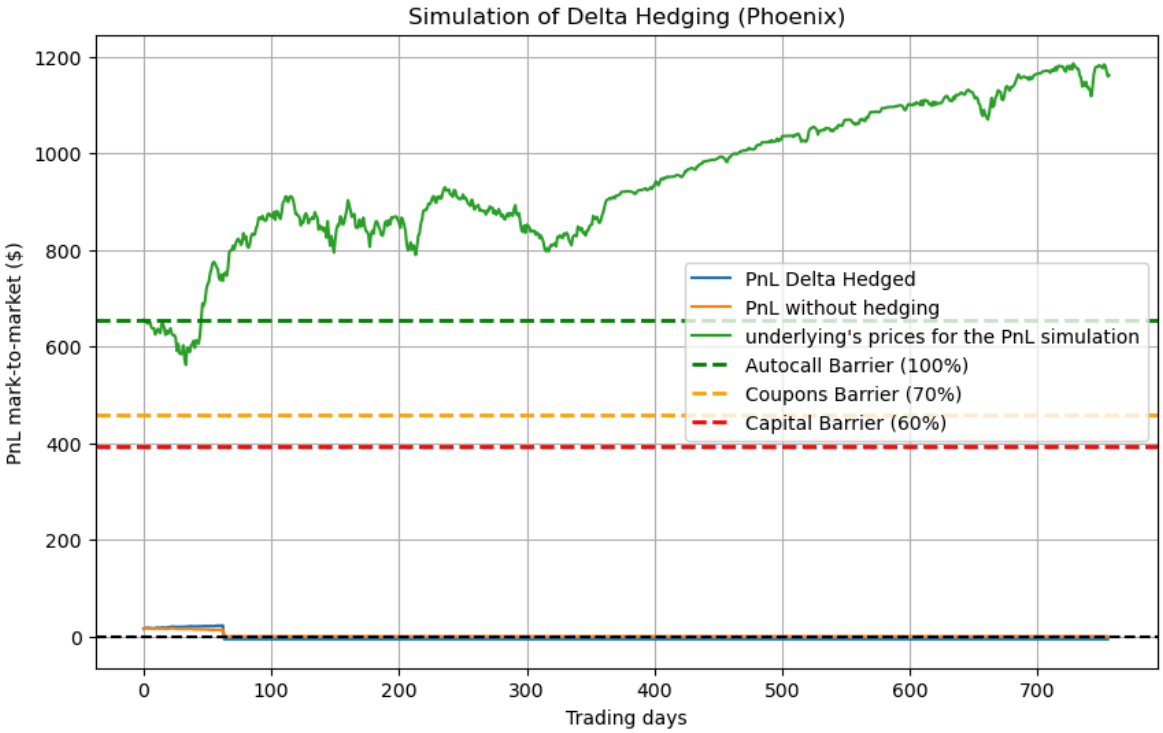
To do so, as it is not a vanilla product, I estimate the delta of the position as the slope of the product's price with respect to the spot, that is :

$$\Delta = \frac{P(S+\epsilon) - P(S)}{\epsilon}$$

where ϵ is a small variation of the spot (e.g. $1\% \times S_t$)

In [155…

```
from src.Analytics import pnl_sim
calibration.barrier_autocall=1

pnl_sim.sim_delta_hedging()
```



The graph illustrates a scenario where the Autocall mechanism is triggered at the very first quarterly observation (day 63). The underlying asset starts exactly at S0 —

the 100% Autocall Barrier — and immediately rallies above it, making early redemption virtually inevitable from inception.

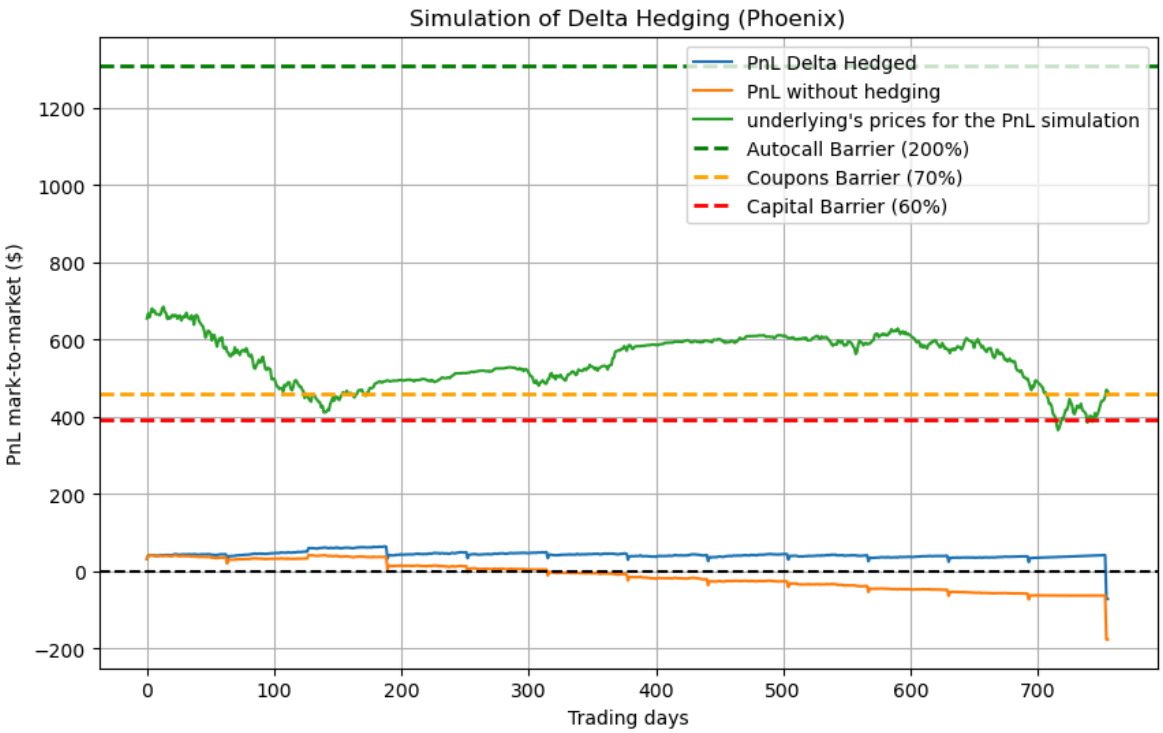
Because the product's life is cut short at the first observation, the delta hedging strategy cannot be meaningfully evaluated: both the hedged and unhedged PnL lines freeze at approximately $0 after day 63, and the underlying continues to be plotted but the product has already terminated.

The near-zero final PnL for the desk reflects a well-functioning hedge over the short period: the autocall payoff (nominal + coupon) was approximately covered by the hedge portfolio. This scenario highlights a structural limitation of Phoenix products for the hedging desk — when the underlying starts at or near the autocall barrier, the product may terminate too early to observe any meaningful hedging dynamics or Gamma monetization.

Let's take a catastrophic scenario to see how the hedge work (with an autocall barrier at 200%) :

In [150…

```
calibration.barrier_autocall=2.0

np.random.seed(123)

pnl_sim.sim_delta_hedging()
```



The simulation illustrates a scenario where the Autocall mechanism is deactivated (barrier set at 200%) to observe the product over its full 3-year maturity. The underlying asset experiences a sharp initial drop from ~$660 to$ 460, hovering near the Coupon Barrier (70%) for most of the product's life.

This trajectory generates an interesting dynamic: on observation dates where the underlying briefly exceeds the coupon barrier, memory coupons are paid out,

creating the visible downward steps in the Unhedged PnL (orange line). Near maturity, the underlying breaches the Capital Barrier (60%), triggering partial capital loss for the client.

The Unhedged PnL (orange) steadily deteriorates from ~+$30 to -250$ by maturity, reflecting the compounding effect of coupon payments and the final capital loss obligation — the bank received $1000 at inception but ultimately paid out more in total cash flows.

In contrast, the Delta-Hedged PnL (blue) remains remarkably stable and positive throughout (~+$30 to +50$), outperforming the unhedged position by approximately $280. This demonstrates the core value of delta hedging: by dynamically rebalancing the hedge portfolio, the desk neutralizes the directional market exposure and preserves the initial structuring margin, regardless of the underlying's adverse trajectory.